BRENT_TTF_regression

March 26, 2026

[1]:
```
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

[2]:
```
def merge_dfs(dfs:list):
    for i in range(len(dfs)):
        if i==0:
            df=dfs[i]
        else:
            df = pd.merge(df, dfs[i], left_index=True, right_index=True)
    return df

def clean_process(df):
    df=df.where(df!=0,np.nan)                                   # - Replace 0 with␣
 ↪NaNs
    df=df.dropna()                                              # - Drop NaNs␣
 ↪(preliminarily)
    df=df.resample('D').asfreq()                                # - Make sure time␣
 ↪axis is even
    df=df.interpolate('linear')                                 # - Fill resulting␣
 ↪NaNs with linear interpolation
    return df

def standardise(col):
    return (col - col.mean()) / col.std()
```

1 Parameters

[15]:
```
# Data Frequency
data_days=28
```

```
frequency=f'{data_days}D' # - Set frequency for resampling in clean_process␣
↪function

indicator_days = 252 # days
indicator_window = round(indicator_days/data_days) # - Window for indicators in␣
↪number of data points
vol_coeff=0.5 # - Coefficient for volatility threshold in indicator definition␣
↪(number of standard deviations above mean)

# Durations
D_util = 7.5 # Utility duration will be higher than Govt duration
D_govt = 8.5

# standardise = True # - Whether to standardise data for comparability of␣
↪coefficients
standardise = False

# Target Definition Selection for regressions
target = ['IGEEUA10 BVLI Index'] # - Utility Debt Yield Index (10Y) as target
# target = ['excess_R_util']  # - Excess Utility Debt Return over Govt Return␣
↪(10Y) as target
```

2 Load, Process, Align Dates

[16]:
```
folder=r'/Users/igorbykov/Desktop/Thesis/data'

# Get Brent data
name=r'commodities_data_v0.xlsx'
brent = pd.read_excel(
    os.path.join(folder,name),
    usecols=[0,1],
    index_col=0
    )
brent.index = pd.to_datetime(brent.index) # - TTF Nat Gas Spot; USD (assumed)
brent = clean_process(brent)

# Get TTF data - EUR
name=r'ttf_front_month.xlsx'
ttf = pd.read_excel(
    os.path.join(folder,name),
    usecols=[0,1],
    index_col=0
    )
ttf.index = pd.to_datetime(ttf.index) # - TTF Nat Gas Spot; USD (assumed)
ttf = clean_process(ttf)
```

```python
# Load Utilities Equity Data - 'SX6P Index' is STOXX600 European Utilities; EUR
name=r'utilities_data_v0.xlsx'
utilities_equity=pd.read_excel(
    os.path.join(folder,name),
    usecols=[0,2],
    index_col=0
    )
utilities_equity.index = pd.to_datetime(utilities_equity.index)     # - Ensure␣
 ↪datatime indexation
utilities_equity = clean_process(utilities_equity)

# Utility debt yield index data (10Y)
name=r'utilities_debt.xlsx'
util_yield_10y=pd.read_excel(
    os.path.join(folder,name),
    usecols=[0,3],
    index_col=0
    )/100                                                           # - Convert␣
 ↪from percentage points to decimals
util_yield_10y.index = pd.to_datetime(util_yield_10y.index)     # - Ensure␣
 ↪datatime indexation
util_yield_10y = clean_process(util_yield_10y)

# Get EUR curve data (10Y)
name=r'euro_curve.xlsx'
govt_yield_10y=pd.read_excel(
    os.path.join(folder,name),
    usecols=[0,2],
    index_col=0
    )/100                                                           # - Convert␣
 ↪from percentage points to decimals
govt_yield_10y.index = pd.to_datetime(govt_yield_10y.index)     # - Ensure␣
 ↪datatime indexation
govt_yield_10y = clean_process(govt_yield_10y)

# Merge all dataframes into one
df = pd.concat([brent,ttf,utilities_equity,util_yield_10y, govt_yield_10y],␣
 ↪axis=1, join='outer').resample(frequency).last()
df.to_excel(os.path.join(folder,'merged_input_data.xlsx'))
df=df.dropna()
```

3 Duration Assumptions, Convert to Log-Returns, Combine

[17]:
```
# Return-Yield calculations account for accrued interest
R_util_debt = np.log((1-D_util * df['IGEEUA10 BVLI Index'].diff() +␣
 ↪df['IGEEUA10 BVLI Index']*data_days/365).dropna() )
R_govt = np.log((1-D_govt * df['10Y'].diff() + df['10Y']*data_days/365).
 ↪dropna() )

R_util_eqty = np.log((df['SX6P Index'] / df['SX6P Index'].shift(1)).dropna() )
R_brent = np.log(df['CO1 Comdty'] / df['CO1 Comdty'].shift(1)).dropna()
R_ttf = np.log((df['TTF_front_month'] / df['TTF_front_month'].shift(1)).
 ↪dropna() )

df = merge_dfs([R_brent, R_ttf, R_util_eqty, R_util_debt, R_govt]).dropna()

df.head(5)
```

[17]:
```
            CO1 Comdty  TTF_front_month  SX6P Index  IGEEUA10 BVLI Index  \
2011-05-16    0.050825        -0.010350   -0.038899             0.009465
2011-06-13   -0.011750        -0.038141   -0.107489             0.004839
2011-07-11   -0.107501         0.051795   -0.067210             0.016051
2011-08-08    0.048181         0.132350    0.132793             0.011687
2011-09-05   -0.082437        -0.007324   -0.043878            -0.010624

                 10Y
2011-05-16  0.008929
2011-06-13  0.006331
2011-07-11  0.025193
2011-08-08  0.023917
2011-09-05  0.015294
```

4 Regime Identification & Standardisation

[18]:
```
mu_p_6m_brent = df['CO1 Comdty'].rolling(window=indicator_window).mean().
 ↪dropna()
std_p_6m_brent = df['CO1 Comdty'].rolling(window=indicator_window).std().
 ↪dropna()
threshold_brent = mu_p_6m_brent + vol_coeff*std_p_6m_brent

mu_p_6m_ttf = df['TTF_front_month'].rolling(window=indicator_window).mean().
 ↪dropna()
std_p_6m_ttf = df['TTF_front_month'].rolling(window=indicator_window).std().
 ↪dropna()
threshold_ttf = mu_p_6m_ttf + vol_coeff*std_p_6m_ttf

# threshold will be shorter than brent due to rolling calculation
```

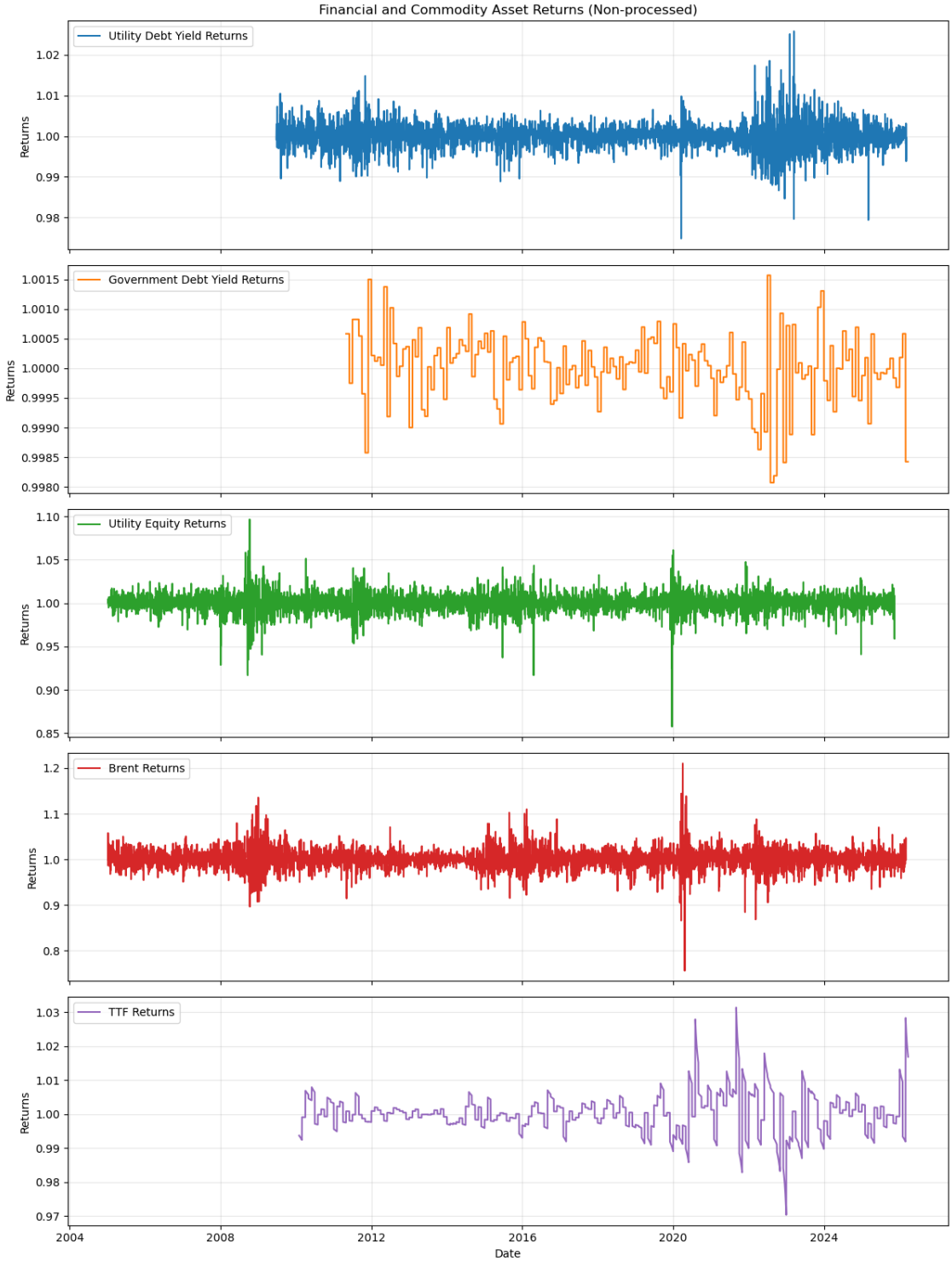
```python
indicator_brent=pd.Series(data=np.where(df['CO1 Comdty'].loc[threshold_brent.
↪index] > threshold_brent, 1, 0), index=threshold_brent.index)
indicator_ttf=pd.Series(data=np.where(df['TTF_front_month'].loc[threshold_ttf.
↪index] > threshold_ttf, 1, 0), index=threshold_ttf.index)

# crop indicator to fit into df
indicator_brent = indicator_brent[indicator_brent.index.isin(df.index)]
indicator_ttf = indicator_ttf[indicator_ttf.index.isin(df.index)]
df['D_brent']=indicator_brent
df['D_ttf']=indicator_ttf

df['govt_indic_brent'] = df['D_brent'] * df['10Y']
df['eqty_indic_brent'] = df['D_brent'] * df['SX6P Index']
df['brent_indic_brent'] = df['D_brent'] * df['CO1 Comdty']

df['govt_indic_ttf'] = df['D_ttf'] * df['10Y']
df['eqty_indic_ttf'] = df['D_ttf'] * df['SX6P Index']
df['ttf_indic_ttf'] = df['D_ttf'] * df['TTF_front_month']

df=df.dropna()

# Standardise all columns to make coefficients comparable
if standardise:
    df=df.apply(lambda col: standardise(col), axis=0)

df['excess_R_util'] = df['IGEEUA10 BVLI Index'] - df['10Y']

df.head(5)
```

[18]:

```
            CO1 Comdty  TTF_front_month  SX6P Index  IGEEUA10 BVLI Index  \
2011-12-26    0.015757         0.008910    0.007217             0.009789
2012-01-23    0.083036         0.045881    0.012254            -0.001285
2012-02-20    0.047640         0.040533   -0.040479             0.015948
2012-03-19   -0.049004         0.013075   -0.035186             0.024108
2012-04-16   -0.068528        -0.026554    0.011951             0.017280

                 10Y  D_brent  D_ttf  govt_indic_brent  eqty_indic_brent  \
2011-12-26  0.014513      0.0    0.0          0.000000          0.000000
2012-01-23  0.006338      1.0    1.0          0.006338          0.012254
2012-02-20  0.006592      1.0    0.0          0.006592         -0.040479
2012-03-19  0.005100      0.0    0.0          0.000000         -0.000000
2012-04-16  0.020333      0.0    0.0          0.000000          0.000000

            brent_indic_brent  govt_indic_ttf  eqty_indic_ttf  ttf_indic_ttf  \
2011-12-26           0.000000        0.000000        0.000000       0.000000
2012-01-23           0.083036        0.006338        0.012254       0.045881
2012-02-20           0.047640        0.000000       -0.000000       0.000000
```

```
2012-03-19          -0.000000        0.000000       -0.000000       0.000000
2012-04-16          -0.000000        0.000000        0.000000      -0.000000

            excess_R_util
2011-12-26      -0.004724
2012-01-23      -0.007623
2012-02-20       0.009355
2012-03-19       0.019007
2012-04-16      -0.003053
```

[19]:
```
series_to_plot = {
    'Utility Debt Yield Returns': (1 - D_util * util_yield_10y.diff()).dropna(),
    'Government Debt Yield Returns': (1 - D_govt * govt_yield_10y.diff()).
 ↪dropna(),
    'Utility Equity Returns': (1 + utilities_equity.pct_change()).dropna(),
    'Brent Returns': (1 + brent.pct_change()).dropna(),
    'TTF Returns': (1 + ttf.pct_change()).dropna()
}

fig, axes = plt.subplots(nrows=len(series_to_plot), ncols=1, figsize=(12, 16),␣
 ↪sharex=True)

for i, (label, data) in enumerate(series_to_plot.items()):
    axes[i].plot(data, label=label, color=f'C{i}') # C0, C1, etc. gives␣
 ↪distinct colors
    axes[i].set_ylabel('Returns')
    axes[i].legend(loc='upper left')
    axes[i].grid(True, alpha=0.3)

axes[0].set_title('Financial and Commodity Asset Returns (Non-processed)')
plt.xlabel('Date')
plt.tight_layout() # Prevents labels from overlapping
plt.show()
```


Financial and Commodity Asset Returns (Non-processed)
Utility Debt Yield Returns
Government Debt Yield Returns
Utility Equity Returns
Brent Returns
TTF Returns
Returns
Date
2004
2008
2012
2016
2020
2024

5 Baseline Unconditional Hedge Model

[20]:
```
max_lags = int(np.floor(4*(len(df)/100)**(2/9))) # - Schwert (1989) rule for␣
↪maximum lags in Newey-West estimator
```

[21]:
```
unconditional_model_brent = sm.OLS(
    df[target],
    sm.add_constant(
        df[['10Y','CO1 Comdty', 'SX6P Index']]
        )
        ).fit(cov_type='HAC',
              cov_kwds={'maxlags': max_lags}
        )

print(unconditional_model_brent.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:     IGEEUA10 BVLI Index   R-squared:                       0.588
Model:                             OLS   Adj. R-squared:                  0.581
Method:                  Least Squares   F-statistic:                     56.74
Date:                 Thu, 26 Mar 2026   Prob (F-statistic):           8.69e-26
Time:                         15:24:14   Log-Likelihood:                 576.13
No. Observations:                  182   AIC:                            -1144.
Df Residuals:                      178   BIC:                            -1131.
Df Model:                            3
Covariance Type:                   HAC
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0004      0.000      0.881      0.378      -0.001       0.001
10Y            0.9793      0.080     12.316      0.000       0.823       1.135
CO1 Comdty     0.0064      0.009      0.745      0.457      -0.010       0.023
SX6P Index     0.0111      0.013      0.832      0.406      -0.015       0.037
==============================================================================
Omnibus:                       43.487   Durbin-Watson:                   2.615
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              432.802
Skew:                          -0.461   Prob(JB):                     1.04e-94
Kurtosis:                      10.498   Cond. No.                         81.5
==============================================================================

Notes:
[1] Standard Errors are heteroscedasticity and autocorrelation robust (HAC)
using 4 lags and without small sample correction
```

[22]:
```
unconditional_model_ttf = sm.OLS(
    df[target],
```

```
    sm.add_constant(
        df[['10Y','TTF_front_month', 'SX6P Index']]
        )
        ).fit(cov_type='HAC',
              cov_kwds={'maxlags': max_lags}
        )
print(unconditional_model_ttf.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:     IGEEUA10 BVLI Index   R-squared:                       0.595
Model:                             OLS   Adj. R-squared:                  0.588
Method:                  Least Squares   F-statistic:                     60.36
Date:                 Thu, 26 Mar 2026   Prob (F-statistic):           5.73e-27
Time:                         15:24:14   Log-Likelihood:                 577.63
No. Observations:                  182   AIC:                            -1147.
Df Residuals:                      178   BIC:                            -1134.
Df Model:                            3
Covariance Type:                   HAC
==================================================================================
===
                      coef    std err          z      P>|z|      [0.025
0.975]
----------------------------------------------------------------------------------
---
const               0.0004      0.001      0.745      0.456      -0.001
0.001
10Y                 0.9761      0.079     12.419      0.000       0.822
1.130
TTF_front_month     0.0109      0.006      1.952      0.051   -4.18e-05
0.022
SX6P Index          0.0134      0.013      1.022      0.307      -0.012
0.039
==============================================================================
Omnibus:                       42.098   Durbin-Watson:                   2.566
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              407.065
Skew:                          -0.434   Prob(JB):                     4.05e-89
Kurtosis:                      10.275   Cond. No.                         80.0
==============================================================================

Notes:
[1] Standard Errors are heteroscedasticity and autocorrelation robust (HAC)
using 4 lags and without small sample correction
```

6 Conditional Hedge Model

[23]:
```
conditional_model_brent = sm.OLS(
    df[target],
    sm.add_constant(
        df[['10Y', 'govt_indic_brent', 'CO1 Comdty', 'brent_indic_brent',␣
 ↪'D_brent', 'SX6P Index', 'eqty_indic_brent']]
        ),
        ).fit(cov_type='HAC',
              cov_kwds={'maxlags': max_lags}
        )
print(conditional_model_brent.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:     IGEEUA10 BVLI Index   R-squared:                       0.599
Model:                             OLS   Adj. R-squared:                  0.583
Method:                  Least Squares   F-statistic:                     30.64
Date:                 Thu, 26 Mar 2026   Prob (F-statistic):           2.34e-27
Time:                         15:24:14   Log-Likelihood:                 578.67
No. Observations:                  182   AIC:                            -1141.
Df Residuals:                      174   BIC:                            -1116.
Df Model:                            7
Covariance Type:                   HAC
==================================================================================
=====
                        coef    std err          z      P>|z|      [0.025
0.975]
----------------------------------------------------------------------------------
-----
const                -0.0002      0.001     -0.354      0.723      -0.002
0.001
10Y                   1.0413      0.095     11.015      0.000       0.856
1.227
govt_indic_brent     -0.2447      0.156     -1.571      0.116      -0.550
0.061
CO1 Comdty            0.0008      0.011      0.076      0.940      -0.020
0.022
brent_indic_brent     0.0030      0.028      0.105      0.916      -0.052
0.058
D_brent               0.0013      0.003      0.459      0.646      -0.004
0.007
SX6P Index            0.0238      0.016      1.489      0.136      -0.008
0.055
eqty_indic_brent     -0.0612      0.038     -1.605      0.109      -0.136
0.014
==============================================================================
```

```
Omnibus:                       45.521   Durbin-Watson:                   2.609
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              379.439
Skew:                          -0.596   Prob(JB):                     4.04e-83
Kurtosis:                       9.972   Cond. No.                         204.
==============================================================================

Notes:
[1] Standard Errors are heteroscedasticity and autocorrelation robust (HAC)
using 4 lags and without small sample correction
```

[24]:
```
conditional_model_ttf = sm.OLS(
    df[target],
    sm.add_constant(
        df[['10Y', 'govt_indic_ttf', 'TTF_front_month', 'ttf_indic_ttf',␣
 ↪'D_ttf', 'SX6P Index', 'eqty_indic_ttf']]
        ),
        ).fit(cov_type='HAC',
              cov_kwds={'maxlags': max_lags}
        )
print(conditional_model_ttf.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:     IGEEUA10 BVLI Index   R-squared:                       0.603
Model:                             OLS   Adj. R-squared:                  0.587
Method:                  Least Squares   F-statistic:                     31.86
Date:                 Thu, 26 Mar 2026   Prob (F-statistic):           3.64e-28
Time:                         15:24:14   Log-Likelihood:                 579.52
No. Observations:                  182   AIC:                            -1143.
Df Residuals:                      174   BIC:                            -1117.
Df Model:                            7
Covariance Type:                   HAC
==================================================================================
===
                      coef    std err          z      P>|z|      [0.025
0.975]
----------------------------------------------------------------------------------
---
const               0.0007      0.001      1.178      0.239      -0.000
0.002
10Y                 0.9352      0.076     12.373      0.000       0.787
1.083
govt_indic_ttf      0.2219      0.269      0.825      0.409      -0.305
0.749
TTF_front_month     0.0171      0.008      2.057      0.040       0.001
0.033
ttf_indic_ttf      -0.0128      0.017     -0.748      0.454      -0.046
0.021
```

```
D_ttf              -0.0003      0.003     -0.085      0.932      -0.006
0.006
SX6P Index          0.0065      0.013      0.491      0.624      -0.020
0.033
eqty_indic_ttf      0.0263      0.038      0.689      0.491      -0.048
0.101
==============================================================================
Omnibus:                       40.954   Durbin-Watson:                   2.562
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              396.210
Skew:                          -0.399   Prob(JB):                     9.20e-87
Kurtosis:                      10.184   Cond. No.                         236.
==============================================================================

Notes:
[1] Standard Errors are heteroscedasticity and autocorrelation robust (HAC)
using 4 lags and without small sample correction
```

7 Hedge Effectiveness

[25]:
```
HE_unconditional_brent = unconditional_model_brent.rsquared
HE_conditional_brent = conditional_model_brent.rsquared

print(f"Brent: Unconditional HE: {HE_unconditional_brent:.4f}")
print(f"Brent: Conditional HE: {HE_conditional_brent:.4f}")
```

```
Brent: Unconditional HE: 0.5882
Brent: Conditional HE: 0.5995
```

[26]:
```
HE_unconditional_ttf = unconditional_model_ttf.rsquared
HE_conditional_ttf = conditional_model_ttf.rsquared

print(f"TTF: Unconditional HE: {HE_unconditional_ttf:.4f}")
print(f"TTF: Conditional HE: {HE_conditional_ttf:.4f}")
```

```
TTF: Unconditional HE: 0.5949
TTF: Conditional HE: 0.6032
```